

Cyrux

Documentação Completa da Base de Dados

Portal de Cibersegurança e Conformidade NIS2 · PostgreSQL · Ano letivo 2025-2026

Ana Caló (30249) · Ana Costa (30218) · Matilde Ângelo (30239) · Rodrigo Albuquerque (30224)

1. Introdução

Este documento descreve **ao pormenor** a base de dados do projeto **Cyrux**, um portal de cibersegurança e conformidade NIS2. A base de dados chama-se `projeto_BD` e é gerida pelo SGBD **PostgreSQL** (administrado através do pgAdmin 4).

A persistência de dados é feita exclusivamente com **SQL puro** (biblioteca `psycopg2` em Python/Django), **sem recurso a ORM**, conforme exigido no enunciado. O esquema é composto por **14 tabelas**, ligadas por **18 chaves estrangeiras**, com **16 índices** de otimização e **4 vistas** (views) que simplificam as consultas.

O documento está organizado em três blocos: (1) as convenções de tipos e restrições usadas; (2) a explicação tabela a tabela, coluna a coluna, de todo o script `schema.sql` (criação e inicialização); (3) o código Python (`basedados.py` e `views.py`) que executa o SQL sobre esta base de dados.

2. Convenções: tipos de dados e restrições

Antes de percorrer as tabelas, importa perceber os **tipos de dados** e as **restrições** (constraints) que se repetem ao longo do script. Compreender estes elementos é o que permite ler qualquer linha do `CREATE TABLE`.

Tipos de dados (PostgreSQL)

Tipo	Significado e uso no projeto
<code>SERIAL</code>	Inteiro que se auto-incrementa a cada inserção. Usado em todas as chaves primárias (<code>id</code>) — o PostgreSQL gera o número automaticamente, garantindo unicidade sem intervenção da aplicação.
<code>INTEGER</code>	Número inteiro. Usado nas chaves estrangeiras (ex.: <code>client_id</code>), que apontam para o <code>id</code> (<code>SERIAL</code>) de

	outra tabela.
SMALLINT	Inteiro pequeno (2 bytes). Usado em campos com valores reduzidos, como <code>sort_order</code> (ordenação) e <code>progress</code> (0–100), poupando espaço em disco.
VARCHAR(n)	Texto com limite máximo de <code>n</code> caracteres. Usado em nomes, emails, títulos e domínios curtos. O limite protege contra dados inesperadamente grandes.
TEXT	Texto sem limite de tamanho. Usado em descrições, conteúdos de artigos, mensagens e notas.
BOOLEAN	Valor lógico <code>TRUE/FALSE</code> . Usado em estados como <code>active</code> , <code>published</code> , <code>read</code> .
DATE	Data (sem hora). Usado em <code>published_at</code> , <code>start_date</code> , <code>deadline</code> , <code>request_date</code> .
TIMESTAMP	Data e hora. Usado em campos de auditoria temporal como <code>created_at</code> , <code>updated_at</code> , <code>upload_date</code> , <code>sent_at</code> .

Restrições (constraints)

Restrição	O que garante
PRIMARY KEY	Identifica de forma única cada registo da tabela. Não pode repetir-se nem ser nula. É a coluna <code>id</code> em

todas as tabelas.

REFERENCES (FK)	Chave estrangeira: obriga o valor a existir noutra tabela (integridade referencial). Ex.: <code>client_id</code> <code>INTEGER REFERENCES users(id)</code> só aceita ids de utilizadores que existam.
NOT NULL	O campo é obrigatório — não pode ficar vazio.
UNIQUE	O valor não se pode repetir na tabela. Ex.: <code>email</code> em <code>users</code> .
CHECK (...)	Regra de validação de domínio. Ex.: <code>CHECK (role IN ('admin', 'manager', 'client'))</code> só permite esses três valores.
DEFAULT	Valor assumido quando a aplicação não fornece nenhum. Ex.: <code>DEFAULT NOW()</code> preenche a data atual; <code>DEFAULT TRUE</code> .
ON DELETE CASCADE	Se o registo "pai" for apagado, os "filhos" que dependem dele são apagados automaticamente. Usado onde a dependência é total (ex.: comentários de um ticket).
ON DELETE SET NULL	Se o "pai" for apagado, a chave estrangeira do "filho" passa a <code>NULL</code> , preservando o registo. Usado para manter histórico (ex.: autor de um documento removido).

Leitura das tabelas. Para cada uma das 14 tabelas mostra-se: o código `CREATE TABLE` exato, uma tabela que explica **coluna a coluna** (tipo, restrições e porquê), e uma nota sobre os dados de inicialização (`INSERT`) que o script insere para demonstração.

3. As tabelas em detalhe

Descreve-se cada tabela pela ordem em que aparece no `schema.sql`. A tabela central de todo o modelo é `users` (utilizadores): quase todas as outras tabelas apontam para ela.

3.1. `users` — Utilizadores

Entidade central do sistema. Guarda todas as contas: administradores, gestores e clientes.

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  role VARCHAR(20) NOT NULL CHECK (role IN ('admin','manager','client')),  
  company VARCHAR(150),  
  twofa_word1 VARCHAR(60),  
  twofa_word2 VARCHAR(60),  
  twofa_word3 VARCHAR(60),  
  active BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

Coluna	Tipo	Explicação
id	SERIAL PK	Identificador único, auto-incrementado. Chave primária referenciada por quase todas as outras tabelas.
name	VARCHAR(100) NOT NULL	Nome do utilizador. Obrigatório.
email	VARCHAR(150) NOT NULL UNIQUE	Email de login. <code>UNIQUE</code> impede dois utilizadores com o mesmo email; <code>NOT NULL</code> torna-o obrigatório.
password_hash	VARCHAR(255) NOT NULL	Palavra-passe guardada com hash bcrypt (nunca em texto simples). 255 caracteres acomodam o hash completo.
role	VARCHAR(20) NOT NULL CHECK	Perfil do utilizador. O <code>CHECK</code> só permite <code>'admin'</code> , <code>'manager'</code> ou <code>'client'</code> , garantindo perfis válidos.
company	VARCHAR(150)	Empresa do cliente. Opcional (só preenchido para clientes).

twofa_word1/2/3	VARCHAR(60)			Três palavras que simulam um mecanismo de dupla autenticação (2FA). Opcionais.
active	BOOLEAN	NOT NULL	DEFAULT TRUE	Indica se a conta está ativa. Por omissão, <code>TRUE</code> . Permite desativar sem apagar.
created_at	TIMESTAMP	NOT NULL	DEFAULT NOW()	Data/hora de criação. Preenchida automaticamente com <code>NOW()</code> .
updated_at	TIMESTAMP	NOT NULL	DEFAULT NOW()	Data/hora da última atualização.

Dados de inicialização. Inserem-se 3 utilizadores de teste. As passwords são hashes bcrypt; as reais para login são: `admin@cyrix.pt` → `admin123`, `manager@cyrix.pt` → `manager123`, `cliente@empresa.pt` → `client123`.

3.2. `team_members` — Membros da equipa

Perfis públicos mostrados na secção "Sobre Nós". Podem existir independentemente de terem conta de utilizador.

```
CREATE TABLE team_members (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    role_label VARCHAR(100) NOT NULL,
    initials VARCHAR(5) NOT NULL,
    user_id INTEGER REFERENCES users(id) ON DELETE SET NULL,
    sort_order SMALLINT NOT NULL DEFAULT 0,
    active BOOLEAN NOT NULL DEFAULT TRUE
);
```

Coluna	Tipo	Explicação
<code>id</code>	SERIAL PK	Identificador único do membro.
<code>name</code>	VARCHAR(100) NOT NULL	Nome do membro da equipa.
<code>role_label</code>	VARCHAR(100) NOT NULL	Cargo apresentado (ex.: "CEO & Fundador", "Diretora Técnica").
<code>initials</code>	VARCHAR(5) NOT NULL	Iniciais usadas para gerar o avatar dinâmico (ex.: "MF").
<code>user_id</code>	INTEGER FK → users	Liga (opcionalmente) a uma conta de utilizador. <code>ON DELETE SET NULL</code> : se o utilizador for apagado, o membro mantém-se como registo histórico, apenas sem ligação.

sort_order	SMALLINT NOT NULL DEFAULT 0	Ordem de apresentação na página.
active	BOOLEAN NOT NULL DEFAULT TRUE	Se o membro está visível.

Dados. 4 membros de equipa (CEO, Diretora Técnica, Gestor de Operações, Consultora Sénior).

3.3. services e service_features — Serviços e funcionalidades

Relação **1-para-Muitos**: cada serviço tem várias funcionalidades. As funcionalidades ficam numa tabela própria (em vez de numa lista dentro de `services`) — isto cumpre a **1.ª Forma Normal** (valores atômicos, sem listas numa célula).

```
CREATE TABLE services (  
    id SERIAL PRIMARY KEY,  
    title VARCHAR(150) NOT NULL,  
    description TEXT NOT NULL,  
    icon VARCHAR(50),  
    active BOOLEAN NOT NULL DEFAULT TRUE,  
    sort_order SMALLINT NOT NULL DEFAULT 0,  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE service_features (  
    id SERIAL PRIMARY KEY,  
    service_id INTEGER NOT NULL REFERENCES services(id) ON DELETE CASCADE,  
    feature VARCHAR(200) NOT NULL,  
    sort_order SMALLINT NOT NULL DEFAULT 0  
);
```

services

Coluna	Tipo	Explicação
id	SERIAL PK	Identificador do serviço.
title	VARCHAR(150) NOT NULL	Nome do serviço (ex.: "Testes de Intrusão").
description	TEXT NOT NULL	Descrição longa do serviço.
icon	VARCHAR(50)	Nome do ícone SVG para o frontend mapear.
active	BOOLEAN DEFAULT TRUE	Se o serviço está ativo/visível.

sort_order	SMALLINT DEFAULT 0	Ordem de apresentação.
created_at	TIMESTAMP DEFAULT NOW()	Data de criação.

service_features

Coluna	Tipo	Explicação
id	SERIAL PK	Identificador da funcionalidade.
service_id	INTEGER NOT NULL FK → services	Serviço a que pertence. ON DELETE CASCADE : se o serviço for removido, as suas funcionalidades são apagadas automaticamente (dependência total).
feature	VARCHAR(200) NOT NULL	Texto da funcionalidade (ex.: "Análise de vulnerabilidades").
sort_order	SMALLINT DEFAULT 0	Ordem dentro do serviço.

Dados. 4 serviços e 16 funcionalidades (4 por serviço).

3.4. articles — Artigos do blog

Notícias e publicações técnicas do portal.

```
CREATE TABLE articles (  
    id SERIAL PRIMARY KEY,  
    title VARCHAR(250) NOT NULL,  
    excerpt TEXT,  
    content TEXT,  
    author VARCHAR(100) NOT NULL,  
    category VARCHAR(80),  
    published BOOLEAN NOT NULL DEFAULT TRUE,  
    published_at DATE,  
    created_by INTEGER REFERENCES users(id) ON DELETE SET NULL,  
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

Coluna	Tipo		Explicação
id	SERIAL	PK	Identificador do artigo.
title	VARCHAR(250)	NOT NULL	Título do artigo.
excerpt	TEXT		Resumo curto mostrado nas listagens.
content	TEXT		Corpo completo do artigo.
author	VARCHAR(100)	NOT NULL	Nome do autor (texto).
category	VARCHAR(80)		Categoria (ex.: "Regulamentação", "Técnico").
published	BOOLEAN	DEFAULT TRUE	Se está publicado ou em rascunho.
published_at	DATE		Data de publicação.
created_by	INTEGER	FK → users	Utilizador que criou o artigo. ON DELETE SET NULL preserva o artigo se o autor for removido.
created_at	/ TIMESTAMP	DEFAULT NOW()	Datas de criação e última atualização.
updated_at			

Dados. 3 artigos sobre NIS2, testes de intrusão e maturidade.

3.5. documents — Documentos de cliente

Ficheiros partilhados de forma privada (por cliente) ou global (públicos).

```
CREATE TABLE documents (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(250) NOT NULL,  
  file_type VARCHAR(20),  
  file_size VARCHAR(20),  
  file_path VARCHAR(500),  
  file_data TEXT,  
  category VARCHAR(80),  
  visibility VARCHAR(20) NOT NULL DEFAULT 'client'  
    CHECK (visibility IN ('global', 'client')),  
  client_id INTEGER REFERENCES users(id) ON DELETE SET NULL,  
  uploaded_by INTEGER REFERENCES users(id) ON DELETE SET NULL,  
  upload_date TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

Coluna	Tipo	Explicação
id	SERIAL PK	Identificador do documento.
name	VARCHAR(250) NOT NULL	Nome do ficheiro.
file_type	VARCHAR(20)	Tipo (PDF, XLSX, DOCX...).
file_size	VARCHAR(20)	Tamanho apresentado (ex.: "2.4 MB").
file_path	VARCHAR(500)	Caminho local de simulação do ficheiro.
file_data	TEXT	Conteúdo do ficheiro em base64 (para upload/download real).
category	VARCHAR(80)	Categoria (Relatórios, Templates, Documentação).
visibility	VARCHAR(20) CHECK	Só permite 'global' ou 'client'. Por omissão 'client' (privado).
client_id	INTEGER FK → users	Cliente dono do documento. Se NULL, o documento é global/público. SET NULL ao apagar o utilizador.
uploaded_by	INTEGER FK → users	Quem carregou o documento.
upload_date	TIMESTAMP DEFAULT NOW()	Data de carregamento.

Dados. 4 documentos (2 privados do cliente 3, 2 globais).

3.6. `service_requests` — Pedidos de serviço

Solicitações submetidas pelos clientes através do portal.

```
CREATE TABLE service_requests (  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(250) NOT NULL,  
  description TEXT,  
  status VARCHAR(30) NOT NULL DEFAULT 'pending'  
    CHECK (status IN ('pending', 'in-progress', 'completed', 'cancelled')),  
  client_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  assigned_to INTEGER REFERENCES users(id) ON DELETE SET NULL,  
  request_date DATE NOT NULL DEFAULT CURRENT_DATE,  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

Coluna	Tipo	Explicação
id	SERIAL PK	Identificador do pedido.
title / description	VARCHAR(250) / TEXT	Título e descrição do pedido.
status	VARCHAR(30) CHECK	Estado do pedido. Só aceita <code>pending</code> , <code>in-progress</code> , <code>completed</code> ou <code>cancelled</code> . Começa em <code>pending</code> .
client_id	INTEGER NOT NULL FK → users	Cliente que fez o pedido. <code>ON DELETE CASCADE</code> : apagar o cliente apaga os seus pedidos.
assigned_to	INTEGER FK → users	Colaborador responsável (opcional). <code>SET NULL</code> ao apagar.
request_date	DATE CURRENT_DATE DEFAULT CURRENT_DATE	Data do pedido (por omissão, hoje).
created_at updated_at	/ TIMESTAMP NOW() DEFAULT NOW()	Datas de criação e atualização.

Dados. 3 pedidos do cliente 3.

3.7. `tickets` e `ticket_comments` — Suporte técnico

Sistema de suporte: cada ticket pode ter vários comentários (relação 1-para-Muitos).

```
CREATE TABLE tickets (  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(250) NOT NULL,
```

```

description TEXT,

category VARCHAR(20) NOT NULL

        CHECK (category IN ('technical','general','incident','billing','other')),

priority VARCHAR(20) NOT NULL DEFAULT 'medium'

        CHECK (priority IN ('low','medium','high','urgent')),

status VARCHAR(20) NOT NULL DEFAULT 'open'

        CHECK (status IN ('open','in-progress','resolved','closed')),

client_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,

assigned_to INTEGER REFERENCES users(id) ON DELETE SET NULL,

created_at TIMESTAMP NOT NULL DEFAULT NOW(),

updated_at TIMESTAMP NOT NULL DEFAULT NOW()

);

CREATE TABLE ticket_comments (

    id SERIAL PRIMARY KEY,

    ticket_id INTEGER NOT NULL REFERENCES tickets(id) ON DELETE CASCADE,

    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    content TEXT NOT NULL,

    created_at TIMESTAMP NOT NULL DEFAULT NOW()

);

```

Coluna	Tipo	Explicação
tickets.id	SERIAL PK	Identificador do ticket.
category	VARCHAR(20) CHECK	Tipo do ticket: technical, general, incident, billing ou other. (Os incidentes — incident — alimentam a métrica "Top 5 incidentes" do dashboard.)
priority	VARCHAR(20) CHECK	Prioridade: low, medium, high ou urgent. Por omissão medium.
status	VARCHAR(20) CHECK	Estado: open, in-progress, resolved ou closed. Por omissão open.
client_id	INTEGER NOT NULL FK	Cliente que abriu o ticket. CASCADE .
assigned_to	INTEGER FK	Técnico responsável. SET NULL .
comments.id	SERIAL PK	Identificador do comentário.
ticket_id	INTEGER NOT NULL FK → tickets	Ticket a que pertence. CASCADE : apagar o ticket apaga os comentários.
user_id	INTEGER NOT NULL	Autor do comentário.

FK → users

content	TEXT NOT NULL	Texto do comentário.
---------	---------------	----------------------

Dados. 3 tickets e 5 comentários.

3.8. `annual_services` — Contratos anuais

Acompanhamento de projetos a longo prazo (conformidade, pentests, formações).

```
CREATE TABLE annual_services (  
    id SERIAL PRIMARY KEY,  
    client_name VARCHAR(150) NOT NULL,  
    client_id INTEGER REFERENCES users(id) ON DELETE SET NULL,  
    service_type VARCHAR(30) NOT NULL  
        CHECK (service_type IN ('pentest', 'nis-compliance', 'maturity-  
assessment', 'training', 'other')),  
    service_name VARCHAR(250) NOT NULL,  
    start_date DATE NOT NULL,  
    deadline DATE NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'pending'  
        CHECK (status IN ('pending', 'in-progress', 'completed', 'overdue', 'cancelled')),  
    progress SMALLINT NOT NULL DEFAULT 0 CHECK (progress BETWEEN 0 AND 100),  
    assigned_to INTEGER REFERENCES users(id) ON DELETE SET NULL,  
    notes TEXT,  
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

Coluna	Tipo	Explicação
id	SERIAL PK	Identificador do contrato.
client_name	VARCHAR(150) NOT NULL	Nome do cliente em texto. Permite guardar clientes que ainda não têm conta (ver nota de desnormalização).
client_id	INTEGER FK → users	Ligação opcional à conta do cliente. <code>SET NULL</code> .
service_type	VARCHAR(30) CHECK	Tipo de contrato: pentest, nis-compliance, maturity-assessment, training ou other. (Os <code>nis-compliance</code> alimentam a métrica de conformidade do dashboard.)
service_name	VARCHAR(250) NOT NULL	Nome do contrato/projeto.
start_date / deadline	DATE NOT NULL	Datas de início e prazo-limite.
status	VARCHAR(20)	Estado: pending, in-progress, completed, overdue ou cancelled.

	CHECK	
progress	SMALLINT CHECK 0–100	Porcentagem de progresso. O <code>CHECK (progress BETWEEN 0 AND 100)</code> impede valores fora do intervalo.
assigned_to	INTEGER FK → users	Colaborador responsável.
notes	TEXT	Observações livres.

Dados. 5 contratos (NIS2, pentest, maturidade, formação), com estados e progressos variados.

3.9. conversations e message_lines — Chat interno

Mensagens diretas entre cliente e staff. Cada conversa tem várias linhas de mensagem (1-para-Muitos).

```
CREATE TABLE conversations (
    id SERIAL PRIMARY KEY,
    client_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    staff_id INTEGER REFERENCES users(id) ON DELETE SET NULL,
    subject VARCHAR(250),
    unread_count SMALLINT NOT NULL DEFAULT 0,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE message_lines (
    id SERIAL PRIMARY KEY,
    conversation_id INTEGER NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    sender_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    content TEXT NOT NULL,
    sent_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

Coluna	Tipo	Explicação
conversations.id	SERIAL PK	Identificador da conversa.
client_id	INTEGER NOT NULL FK	Cliente da conversa. <code>CASCADE</code> .
staff_id	INTEGER FK	Colaborador que responde. <code>SET NULL</code> .
subject	VARCHAR(250)	Assunto da conversa.

unread_count	SMALLINT DEFAULT 0	Contador de mensagens não lidas.
message_lines.id	SERIAL PK	Identificador da mensagem.
conversation_id	INTEGER NOT NULL FK	Conversa a que pertence. CASCADE .
sender_id	INTEGER NOT NULL FK → users	Quem enviou a mensagem.
content	TEXT NOT NULL	Texto da mensagem.
sent_at	TIMESTAMP DEFAULT NOW()	Data/hora de envio (para ordenação cronológica).

Dados. 1 conversa e 4 mensagens.

3.10. contact_submissions — Contactos públicos

Mensagens vindas do formulário público da landing page. **Não tem chave estrangeira** para **users**: são leads de visitantes não autenticados, propositadamente isolados.

```
CREATE TABLE contact_submissions (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  company VARCHAR(150),
  phone VARCHAR(30),
  message TEXT NOT NULL,
  read BOOLEAN NOT NULL DEFAULT FALSE,
  reply TEXT,
  replied_at TIMESTAMP,
  submitted_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

Coluna	Tipo	Explicação
id	SERIAL PK	Identificador da submissão.
name / email	VARCHAR NOT NULL	Nome e email do visitante (obrigatórios).
company / phone	VARCHAR	Empresa e telefone (opcionais).
message	TEXT NOT NULL	Mensagem enviada.
read	BOOLEAN DEFAULT FALSE	Se o staff já leu.
reply / replied_at	TEXT / TIMESTAMP	Resposta do staff e respetiva data.
submitted_at	TIMESTAMP DEFAULT NOW()	Data de submissão.

Dados. Tabela criada vazia — preenchida em runtime pelo formulário (função `inserir_contacto`).

3.11. `audit_log` — Registo de auditoria

Rastreabilidade de eventos e ações do sistema (logins, alterações, alertas de segurança).

```
CREATE TABLE audit_log (  
    id SERIAL PRIMARY KEY,  
    action VARCHAR(250) NOT NULL,  
    category VARCHAR(30) NOT NULL  
    CHECK (category IN  
( 'authentication', 'content', 'users', 'documents', 'system', 'security' )),  
    severity VARCHAR(20) NOT NULL DEFAULT 'info'  
    CHECK (severity IN ( 'info', 'warning', 'critical' )),  
    user_id INTEGER REFERENCES users(id) ON DELETE SET NULL,  
    user_email VARCHAR(150),  
    ip_address VARCHAR(45),  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

Coluna	Tipo	Explicação
<code>id</code>	SERIAL PK	Identificador do registo.
<code>action</code>	VARCHAR(250) NOT NULL	Descrição da ação (ex.: "Login realizado").
<code>category</code>	VARCHAR(30) CHECK	Categoria do evento: authentication, content, users, documents, system ou security.
<code>severity</code>	VARCHAR(20) CHECK	Gravidade: info, warning ou critical. Por omissão info.
<code>user_id</code>	INTEGER FK → users	Autor da ação. <code>SET NULL</code> se o utilizador for apagado.
<code>user_email</code>	VARCHAR(150)	Email do autor guardado em texto (mantém-se mesmo após remoção do utilizador — ver desnormalização).
<code>ip_address</code>	VARCHAR(45)	IP de origem (45 caracteres suportam IPv6).
<code>created_at</code>	TIMESTAMP DEFAULT NOW()	Data/hora do evento.

Dados. 10 registos de auditoria de exemplo.

Nota de desenho — desnormalização justificada. Em `annual_services` guarda-se `client_name` além do `client_id`, e em `audit_log` guarda-se `user_email` além do `user_id`. É uma opção consciente: preservar o nome/email **histórico** mesmo que o utilizador seja removido, e (no caso dos contratos) permitir

registar clientes sem conta. Sem esta escolha, ao apagar o utilizador perder-se-ia essa informação.

4. Índices de otimização

Um **índice** é uma estrutura auxiliar que acelera as pesquisas por uma coluna, evitando percorrer a tabela inteira. Criam-se índices nas colunas usadas com frequência em `WHERE`, `JOIN` e `ORDER BY`. O projeto define 16 índices:

```
CREATE INDEX idx_documents_client      ON documents(client_id);
CREATE INDEX idx_documents_visibility  ON documents(visibility);
CREATE INDEX idx_requests_client       ON service_requests(client_id);
CREATE INDEX idx_requests_status       ON service_requests(status);
CREATE INDEX idx_tickets_client        ON tickets(client_id);
CREATE INDEX idx_tickets_status        ON tickets(status);
CREATE INDEX idx_tickets_priority      ON tickets(priority);
CREATE INDEX idx_ticket_comments_ticket ON ticket_comments(ticket_id);
CREATE INDEX idx_annual_status         ON annual_services(status);
CREATE INDEX idx_annual_assigned       ON annual_services(assigned_to);
CREATE INDEX idx_audit_severity        ON audit_log(severity);
CREATE INDEX idx_audit_created         ON audit_log(created_at DESC);
CREATE INDEX idx_audit_user            ON audit_log(user_id);
CREATE INDEX idx_articles_published   ON articles(published_at DESC);
CREATE INDEX idx_messages_conversation ON message_lines(conversation_id);
CREATE INDEX idx_conversations_client  ON conversations(client_id);
```

Índice	Porquê
idx_documents_client / _visibility	Filtrar documentos por cliente e por visibilidade (privado/global).
idx_requests_client / _status	Listar pedidos por cliente e filtrar por estado.
idx_tickets_client / _status / _priority	As três dimensões mais consultadas dos tickets (por cliente, estado e prioridade).
idx_ticket_comments_ticket	Carregar rapidamente os comentários de um ticket.
idx_annual_status / _assigned	Filtrar contratos por estado e por responsável.
idx_audit_severity / _created / _user	Consultar logs por gravidade, por data (<code>DESC</code> = mais recentes primeiro) e por autor.
idx_articles_published	Listar artigos por data de publicação decrescente.
idx_messages_conversation	/ Carregar mensagens de uma conversa e conversas de um cliente.

5. Vistas relacionais (VIEWS)

Uma **vista** é uma consulta guardada, apresentada como se fosse uma tabela. Encapsula `JOINS` e filtros complexos, simplificando o código da aplicação. O projeto tem 4 vistas.

5.1. `vw_open_tickets` — Tickets abertos com dados do cliente

```
CREATE OR REPLACE VIEW vw_open_tickets AS
SELECT t.id, t.title, t.category, t.priority, t.status, t.created_at,
       c.name AS client_name, c.email AS client_email, c.company AS client_company,
       a.name AS assigned_to,
       (SELECT COUNT(*) FROM ticket_comments tc WHERE tc.ticket_id = t.id) AS comment_count
FROM tickets t
JOIN users c ON c.id = t.client_id
LEFT JOIN users a ON a.id = t.assigned_to
WHERE t.status NOT IN ('resolved', 'closed');
```

Junta cada ticket com o **cliente** que o abriu (`JOIN users c`) e com o **responsável** (`LEFT JOIN users a` — `LEFT` porque pode não haver responsável). A subconsulta conta os comentários de cada ticket. O `WHERE` mostra apenas tickets que **não** estão resolvidos nem fechados, ou seja, os que ainda precisam de atenção.

5.2. `vw_documents` — Documentos com nomes de cliente e autor

```
CREATE OR REPLACE VIEW vw_documents AS
SELECT d.id, d.name, d.file_type, d.file_size, d.category, d.visibility, d.upload_date,
       u.name AS client_name, u.email AS client_email, up.name AS uploaded_by_name
FROM documents d
LEFT JOIN users u ON u.id = d.client_id
LEFT JOIN users up ON up.id = d.uploaded_by;
```

Substitui os ids numéricos (`client_id`, `uploaded_by`) pelos **nomes legíveis** do cliente e de quem carregou. Usa `LEFT JOIN` porque um documento global pode não ter cliente associado (`client_id NULL`).

5.3. vw_annual_services — Contratos com alerta de prazo

```
CREATE OR REPLACE VIEW vw_annual_services AS
SELECT s.id, s.client_name, s.service_type, s.service_name, s.start_date, s.deadline,
       s.status, s.progress, s.notes, u.name AS assigned_to_name,
       CASE WHEN s.deadline < CURRENT_DATE AND s.status NOT IN ('completed','cancelled')
            THEN TRUE ELSE FALSE END AS is_overdue
FROM annual_services s
LEFT JOIN users u ON u.id = s.assigned_to;
```

Além de juntar o nome do responsável, calcula em tempo real um campo `is_overdue`: com um `CASE`, marca como atrasado (`TRUE`) qualquer contrato cujo `deadline` já passou e que ainda não está concluído nem cancelado. É lógica de negócio resolvida diretamente em SQL.

5.4. vw_audit_alerts — Alertas de auditoria

```
CREATE OR REPLACE VIEW vw_audit_alerts AS
SELECT a.id, a.action, a.category, a.severity, a.user_email, a.ip_address, a.created_at,
       u.name AS user_name
FROM audit_log a
LEFT JOIN users u ON u.id = a.user_id
WHERE a.severity IN ('warning','critical')
ORDER BY a.created_at DESC;
```

Filtra o registo de auditoria para mostrar apenas os eventos **importantes** (`warning` e `critical`), ordenados do mais recente para o mais antigo. Serve de painel de segurança.

6. O código que usa a base de dados (`basedados.py`)

A aplicação Django acede à base de dados através do ficheiro `portal/basedados.py`, que usa **SQL direto** com a biblioteca `psycopg2` — **não há ORM**. Cada operação é uma instrução SQL escrita à mão. Esta secção explica cada função.

6.1. Ligação à base de dados

```
import psycopg2

from psycopg2.extras import RealDictCursor

import bcrypt

def obter_conexao():

    return psycopg2.connect(

        dbname="projeto_BD", user="postgres", password="BDCatarina6",

        host="localhost", port="5432"

    )
```

`obter_conexao()` abre uma ligação ao servidor PostgreSQL local (porta 5432), à base `projeto_BD`. Importa-se `bcrypt` para verificar palavras-passe e `RealDictCursor` para que os resultados venham como dicionários (`{coluna: valor}`) em vez de tuplos — facilita o uso no template.

6.2. Utilitários de leitura e escrita

```
def executar_consulta(query, params=None):

    conn = obter_conexao()

    cursor = conn.cursor(cursor_factory=RealDictCursor)

    try:

        cursor.execute(query, params)

        return cursor.fetchall()

    except Exception as e:

        print(f"Erro ao executar consulta SQL: {e}")

        return []

    finally:

        cursor.close(); conn.close()
```

```
def executar_comando(query, params=None):

    conn = obter_conexao()

    cursor = conn.cursor()

    try:

        cursor.execute(query, params)

        conn.commit()

        return True

    except Exception as e:

        print(f"Erro ao executar comando SQL: {e}")

        conn.rollback(); return False

    finally:

        cursor.close(); conn.close()
```

Duas funções centralizam o acesso a SQL, evitando repetir código:

`executar_consulta()` serve para **SELECT**: executa a query, devolve **todas** as linhas (`fetchall`) como lista de dicionários. Em caso de erro, devolve lista vazia. O `finally` garante que a ligação é sempre fechada.

`executar_comando()` serve para **INSERT/UPDATE/DELETE**: executa e faz `commit()` (confirma a escrita). Se falhar, faz `rollback()` (desfaz) e devolve `False`. O parâmetro `params` é passado separadamente da query — isto é **SQL parametrizado**, que protege contra injeção de SQL.

6.3. As 5 métricas do dashboard (Ficha 9)

Cada métrica é uma consulta SQL de agregação (`GROUP BY`, `COUNT`, `AVG`). Os nomes das colunas resultantes (aliases com `AS`) são os que o template espera.

Métrica 1 — Conformidade NIS2 por estado

```
SELECT status AS estado_nis2, COUNT(*) AS total

FROM annual_services

WHERE service_type = 'nis-compliance'

GROUP BY status

ORDER BY total DESC;
```

Conta os contratos de conformidade NIS2 (`service_type = 'nis-compliance'`) agrupados por estado. Resultado: quantos contratos há em cada estado (completed, overdue, etc.).

Métrica 2 — Top 5 clientes com mais incidentes

```
SELECT u.name AS nome, COUNT(t.id) AS total_incidentes

FROM users u
```

```
JOIN tickets t ON t.client_id = u.id

WHERE t.category = 'incident'

GROUP BY u.id, u.name

ORDER BY total_incidentes DESC

LIMIT 5;
```

Junta utilizadores com os seus tickets, filtra os de categoria `incident`, conta por cliente e ordena do maior para o menor, limitando a 5 (`LIMIT 5`).

Métrica 3 — Documentos por cliente e por mês

```
SELECT u.name AS cliente,

       TO_CHAR(d.upload_date, 'YYYY-MM') AS mes,

       COUNT(d.id) AS total_documentos

FROM documents d

JOIN users u ON u.id = d.client_id

GROUP BY u.name, mes

ORDER BY mes DESC, total_documentos DESC;
```

`TO_CHAR(d.upload_date, 'YYYY-MM')` transforma a data no formato ano-mês. Agrupa por cliente e por mês, contando os documentos submetidos em cada combinação.

Métrica 4 — Distribuição de utilizadores por perfil

```
SELECT CASE role

       WHEN 'admin'   THEN 'Administrador'

       WHEN 'manager' THEN 'Colaborador'

       WHEN 'client'  THEN 'Cliente'

       ELSE role

       END AS perfil,

       COUNT(*) AS total

FROM users

GROUP BY role

ORDER BY total DESC;
```

Conta os utilizadores por `role`. O `CASE` traduz os códigos internos (admin/manager/client) para etiquetas legíveis em português, apresentadas diretamente no dashboard.

Métrica 5 — Estado dos tickets e tempo médio de resolução

```
SELECT status AS estado,

       COUNT(*) AS total_tickets,
```

```
ROUND(AVG(EXTRACT(EPOCH FROM (updated_at - created_at)) / 86400.0)::numeric, 1)

        AS tempo_medio_resolucao

FROM tickets

GROUP BY status

ORDER BY total_tickets DESC;
```

Agrupar os tickets por estado e contá-los. Para o tempo médio: `updated_at - created_at` dá a duração; `EXTRACT(EPOCH ...)` converte-a em segundos; dividir por `86400` dá dias; `AVG` calcula a média e `ROUND(...,1)` arredonda a 1 casa decimal.

6.4. Autenticação com bcrypt

```
def validar_login(email, password):
    conn = obter_conexao()

    cursor = conn.cursor(cursor_factory=RealDictCursor)

    try:
        cursor.execute(
            "SELECT id, name, email, role, password_hash "
            "FROM users WHERE email = %s AND active = TRUE;", (email,))

        utilizador = cursor.fetchone()

        if not utilizador:
            return None

        hash_guardado = utilizador['password_hash']

        if not bcrypt.checkpw(password.encode('utf-8'), hash_guardado.encode('utf-8')):
            return None

        return {'id': utilizador['id'], 'nome': utilizador['name'],
                'email': utilizador['email'],
                'perfil': ROLE_PARA_PERFIL.get(utilizador['role'], utilizador['role'])}

    finally:
        cursor.close(); conn.close()
```

O login procura o utilizador pelo `email` (e só contas `active = TRUE`). Como a palavra-passe está guardada com **hash bcrypt**, não se compara texto com texto: usa-se `bcrypt.checkpw()`, que verifica se a password introduzida corresponde ao hash guardado. Se corresponder, devolve os dados do utilizador com o perfil já traduzido para português; caso contrário, devolve `None` (credenciais inválidas).

6.5. CRUD de utilizadores

```
def criar_utilizador(nome, email, password, perfil):
    role = PERFIL_PARA_ROLE.get(perfil, perfil)

    password_hash = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')

    return executar_comando(
        "INSERT INTO users (name, email, password_hash, role) VALUES (%s, %s, %s, %s);",
        (nome, email, password_hash, role))

def atualizar_utilizador(id_utilizador, nome, email, perfil):
```

```

    role = PERFIL_PARA_ROLE.get(perfil, perfil)

    return executar_comando(

        "UPDATE users SET name=%s, email=%s, role=%s, updated_at=NOW() WHERE id=%s;",

        (nome, email, role, id_utilizador))

def eliminar_utilizador(id_utilizador):

    return executar_comando("DELETE FROM users WHERE id = %s;", (id_utilizador,))

```

As três operações de escrita sobre `users`:

CREATE — `criar_utilizador`: converte o perfil legível para o código interno (`role`), gera o hash bcrypt da password com `bcrypt.hashpw` + `gensalt()` (nunca guarda texto simples) e faz o `INSERT`.

UPDATE — `atualizar_utilizador`: altera nome, email e perfil de um utilizador, atualizando também `updated_at`.

DELETE — `eliminar_utilizador`: remove o utilizador pelo `id`. (Graças às regras `ON DELETE`, os registos dependentes são tratados automaticamente.)

6.6. Formulário de contacto

```

def inserir_contacto(nome, email, mensagem):

    return executar_comando(

        "INSERT INTO contact_submissions (name, email, message) VALUES (%s, %s, %s);",

        (nome, email, mensagem))

```

Grava uma submissão do formulário público na tabela `contact_submissions`. É a operação de escrita mais simples do projeto.

7. A ligação Django (`views.py`)

```

def home(request):

    if request.method == 'POST':

        data = json.loads(request.body)

        action = data.get('action')

        if action == 'contacto':

            sucesso = basedados.inserir_contacto(data.get('name'), data.get('email'),
data.get('message'))

            return JsonResponse({'status': 'success' if sucesso else 'error'})

        elif action == 'login':

```

```

        user = basedados.validar_login(data.get('email'), data.get('password'))

        ...

# GET: calcula as 5 metricas e envia-as ao template

dashboard = {

    'conformidade': basedados.obter_conformidade_nis2(),

    'incidentes':    basedados.obter_top5_incidentes(),

    'documentos':    basedados.obter_documentos_por_mes(),

    'perfis':        basedados.obter_distribuicao_perfis(),

    'tickets':       basedados.obter_estado_tickets_resolucao(),

}

contexto = {'dashboard_json': json.dumps(dashboard, default=str)}

return render(request, 'portal/cyrix.html', contexto)

```

A [view home](#) é o ponto de entrada. Em pedidos **POST** (assíncronos, vindos do frontend) despacha a ação: [contacto](#), [login](#) ou as operações de CRUD, chamando a função correspondente do [basedados.py](#) e devolvendo [JsonResponse](#). Em pedidos **GET** (carregamento da página) executa as 5 consultas do dashboard, junta-as num dicionário e serializa-o em JSON ([json.dumps](#), com [default=str](#) para tipos como datas/decimais), enviando-o ao template [cyrix.html](#), onde o dashboard os apresenta.

8. Fluxo resumido

Juntando tudo, o percurso de um pedido é:

```

Browser → views.py → basedados.py (SQL via psycopg2) → PostgreSQL

                        ↑

                        bcrypt (verificação de password)

```

O browser envia o pedido; a [view](#) decide a ação; o [basedados.py](#) executa a instrução SQL correspondente sobre a base [projeto_BD](#); o PostgreSQL responde; e o resultado volta ao browser (JSON ou página renderizada). Toda a persistência é SQL puro, sem ORM, cumprindo o requisito do enunciado.

Resumo. 14 tabelas com integridade garantida por chaves primárias, 18 chaves estrangeiras e regras ON DELETE; domínios validados por CHECK; 16 índices para desempenho; 4 vistas para encapsular consultas; e uma camada Python que executa SQL parametrizado e valida palavras-passe com [bcrypt](#). O modelo está normalizado (até à 3.ª Forma Normal), com desnormalizações pontuais devidamente justificadas para preservar histórico.